Complete Numpy Basics

NumPy for Data Science — Complete Cheat Sheet (Everything at a Glance)

NumPy (Numerical Python) is the **foundation of Data Science, Machine Learning, Deep Learning, Pandas, Scikit-Learn, TensorFlow, and PyTorch**.

---

1. Introduction to NumPy

What is NumPy?

NumPy is a Python library used for:

- Fast numerical computations
- Multi-dimensional arrays
- Mathematical operations
- Linear algebra
- Statistics

```
Python
import numpy as np
```

NumPy vs Python Lists

Feature	Python List	NumPy Array
Speed	Slow	Very Fast
Memory	More	Less
Vectorized Operations	No	Yes
Mathematical Functions	Limited	Extensive

```
Python
lst = [1,2,3]
arr = np.array([1,2,3])
```

ndarray Basics

```
Python
a = np.array([1,2,3,4])
```

Array Attributes

```python
a.shape      # (4,)
a.size       # 4
a.ndim       # 1
a.dtype      # int64
```

2. Array Creation

Create Array

```python
np.array([1,2,3])
```

Zeros

```python
np.zeros((2,3))
```

Ones

```python
np.ones((3,2))
```

Full

```python
np.full((2,2), 10)
```

Empty

```python
np.empty((2,2))
```

Arange

```python
np.arange(0,10,2)
```

Output:

```python
[0 2 4 6 8]
```

Linspace

```python
np.linspace(0,1,5)
```

Output:

```
Python
[0.   0.25 0.5 0.75 1.]
```

Identity Matrix

```
Python
np.eye(3)
```

Random Arrays

```
Python
np.random.rand(3,3)
np.random.randint(1,10,(2,2))
```

3. Array Indexing and Slicing

Basic Indexing

```
Python
a = np.array([10,20,30,40])

a[0]
a[2]
```

Negative Indexing

```
Python
a[-1]
```

Slicing

```
Python
a[1:4]
a[:3]
a[2:]
```

2D Indexing

```
Python
arr = np.array([[1,2],
                [3,4]])

arr[0,1]
```

Output:

```
Python
2
```

Boolean Indexing

```
Python
a[a > 20]
```

Fancy Indexing

```
Python
a[[0,2,3]]
```

4. Array Manipulation

Reshape

```
Python
a.reshape(2,3)
```

Flatten

```
Python
a.flatten()
```

Ravel

```
Python
a.ravel()
```

Transpose

```
Python
a.T
np.transpose(a)
```

Resize

```
Python
np.resize(a,(3,4))
```

Insert

```
Python
np.insert(a,1,99)
```

Delete

```
Python
np.delete(a,2)
```

Append

```python
np.append(a,100)
```

5. Mathematical Operations

Arithmetic Operations

```python
a+b
a-b
a*b
a/b
```

Scalar Operations

```python
a+10
a*2
```

Comparison Operations

```python
a > 5
a == 3
```

Logical Operations

```python
np.logical_and(a>2, a<8)
```

Mathematical Functions

```python
np.sqrt(a)
np.exp(a)
np.log(a)

np.sin(a)
np.cos(a)
np.tan(a)

np.abs(a)
np.round(a)
```

6. Broadcasting

Rule 1

Dimensions must be equal OR one of them should be 1.

Scalar Broadcasting

```python
a + 10
```

Array Broadcasting

```python
A = np.array([[1],[2],[3]])

B = np.array([10,20,30])

A + B
```

Output:

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

7. Aggregation Functions

Sum

```python
np.sum(a)
```

Mean

```python
np.mean(a)
```

Median

```python
np.median(a)
```

Standard Deviation

```python
np.std(a)
```

Variance

```python
np.var(a)
```

Minimum

```python
np.min(a)
```

Maximum

```python
np.max(a)
```

Argmin

```python
np.argmin(a)
```

Argmax

```python
np.argmax(a)
```

Axis-Based Operations

```python
arr.sum(axis=0)
arr.sum(axis=1)
```

8. Statistical Operations

Mean

```python
np.mean(data)
```

Median

```python
np.median(data)
```

Mode

NumPy does not provide mode directly.

Use:

```python
from scipy.stats import mode
```

Variance

```python
```

```
np.var(data)
```

Standard Deviation

```
Python
np.std(data)
```

Percentiles

```
Python
np.percentile(data,50)
```

Quantiles

```
Python
np.quantile(data,0.75)
```

Correlation

```
Python
np.corrcoef(x,y)
```

Covariance

```
Python
np.cov(x,y)
```

9. Random Module

Random Float

```
Python
np.random.rand()
```

Random Integers

```
Python
np.random.randint(1,100)
```

Random Sampling

```
Python
np.random.sample(5)
```

Random Choice

```
Python
```

```
np.random.choice([1,2,3,4], size=3)
```

Shuffle

```
Python
np.random.shuffle(a)
```

Seed

```
Python
np.random.seed(42)
```

Normal Distribution

```
Python
np.random.normal(
    loc=0,
    scale=1,
    size=1000
)
```

10. Sorting, Searching & Filtering

Sort

```
Python
np.sort(a)
```

Argsort

Returns indices.

```
Python
np.argsort(a)
```

Where

```
Python
np.where(a>5)
```

Unique

```
Python
np.unique(a)
```

Filtering

```
Python
a[a>5]
```

Conditional Selection

```python
np.where(a>5, "Yes", "No")
```

11. Combining and Splitting Arrays

Concatenate

```python
np.concatenate((a,b))
```

Vertical Stack

```python
np.vstack((a,b))
```

Horizontal Stack

```python
np.hstack((a,b))
```

Split

```python
np.split(a,2)
```

12. Handling Missing Values

NaN

```python
np.nan
```

Check Missing Values

```python
np.isnan(a)
```

Ignore NaN

```python
np.nanmean(a)
np.nansum(a)

np.nanmax(a)
np.nanmin(a)
```

13. Linear Algebra

Vector

```python
v = np.array([1,2,3])
```

Matrix

```python
A = np.array([[1,2],[3,4]])
```

Dot Product

```python
np.dot(a,b)
```

Matrix Multiplication

```python
A @ B

# OR

np.matmul(A,B)
```

Determinant

```python
np.linalg.det(A)
```

Inverse

```python
np.linalg.inv(A)
```

Eigenvalues & Eigenvectors

```python
np.linalg.eig(A)
```

Solve Linear Equation

```python
A = np.array([[2,1],
              [1,3]])

B = np.array([8,13])

np.linalg.solve(A,B)
```

14. File Operations

Save Binary File

```Python
np.save("data.npy", a)
```

Load Binary File

```Python
np.load("data.npy")
```

Save Text File

```Python
np.savetxt("data.txt", a)
```

Load Text File

```Python
np.loadtxt("data.txt")
```

15. Performance Optimization

Why NumPy is Fast?

Vectorization

Bad:

```Python
result = []

for i in data:
    result.append(i*2)
```

Good:

```Python
result = data * 2
```

Memory Efficiency

NumPy stores data in contiguous memory blocks.

Python lists store references.

Hence NumPy consumes less memory.

Avoid Loops

Always prefer:

```python
np.sum(a)
np.mean(a)
a*10
```

instead of manual loops.

---

16. Real Data Analysis Practice

Student Marks Analysis

```python
marks = np.array([78,65,89,90,45])

np.mean(marks)
np.max(marks)
np.min(marks)
```

Sales Analysis

```python
sales = np.array([1000,1200,900,1500])

sales.sum()
sales.mean()
```

Weather Analysis

```python
temp = np.array([32,34,30,29,35])

np.mean(temp)
np.std(temp)
```

Dataset Cleaning

```python
data = np.array([10,20,np.nan,30])

np.nanmean(data)
```

Statistical Summary

```python
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Std:", np.std(data))
print("Min:", np.min(data))
print("Max:", np.max(data))
```

What You Must Master for Data Science

1. Array Creation
2. Indexing & Slicing
3. Reshaping
4. Broadcasting
5. Mathematical Operations
6. Aggregation Functions
7. Statistical Functions
8. Random Module
9. Sorting & Filtering
10. Missing Values
11. Linear Algebra
12. File Operations
13. Vectorization & Optimization
14. Real Dataset Analysis

If you master these 14 areas and solve 30–40 NumPy practice problems, you'll have essentially all the NumPy knowledge required for Data Science, Machine Learning, and Deep Learning.